

ROS 2 Practical Assignment:

SO101 Motion Planning & Perception Integration

System Architecture Explanation

1. Overview

This project implements a complete perception-driven robotic manipulation pipeline in Isaac Sim using ROS 2. The system autonomously detects a red cup, plans a motion toward it, grasps it, and places it inside a box while ignoring other objects in the scene.

The architecture follows a modular design with clear separation between perception, planning, execution, and high-level decision-making. Communication between modules is handled using ROS 2 topics.

2. System Components

2.1. Perception Module (`perception_node`)

The perception module is responsible for detecting the red cup and estimating its position in the robot's base frame.

- Inputs:
 - RGB image (`/rgb`)
 - Depth image (`/depth`)
 - Camera info (`/camera_info`)
- Process:
 - Color-based segmentation is used to detect the red cup in the RGB image
 - Depth data is used to estimate the 3D position of the detected object
 - A static transform (`camera_link` → `base_link`) converts the pose into the robot's base frame
- Output:
 - `/red_cup_pose` (`geometry_msgs/PoseStamped`)

2.2. Motion Planning Module (`movegroup_executor`)

This module is responsible for generating and executing robot motions using MoveIt.

- Uses:

- MoveIt planning interface
- Robot joint states (`/joint_states`)
- Responsibilities:
 - Convert perception output into target poses
 - Plan trajectories using MoveIt
 - Execute trajectories by publishing to `/planned_arm_trajectory`
 - Control gripper states

2.3. Trajectory Execution (`trajectory_to_isaac`)

This node converts planned trajectories into commands compatible with Isaac Sim.

- Input:
 - `/planned_arm_trajectory`
- Output:
 - `/isaac_joint_command`

It ensures that MoveIt-generated trajectories are correctly executed in simulation.

2.4. State Relay (`isaac_state_relay`)

This module synchronizes simulation joint states with ROS 2.

- Input:
 - `/isaac_joint_states`
- Output:
 - `/joint_states`

This is essential for MoveIt, which relies on real-time joint feedback for planning and execution.

2.5. Behavior Tree (`bt_node`)

The Behavior Tree provides high-level task orchestration. It sequences the entire manipulation pipeline:

Wait for detection → safe_home → open → plan_to_cup → align_to_grasp → close → attach → lift_cup → above_box → drop_in_box → open → detach → above_box → safe_home

3. Grasping Strategy

Due to limitations of rigid-body physics attachment in Isaac Sim, a kinematic follow-based attachment approach is used:

- At grasp time:
 - The relative transform between the cup and gripper is captured
 - Cup physics is disabled
- During motion:
 - Cup pose is continuously updated relative to the gripper

4. System Workflow

- Camera detects red cup
- Perception node publishes cup pose
- Behavior Tree triggers planning
- MoveIt generates trajectory
- Trajectory executed in Isaac Sim
- Gripper closes and attaches object
- Object is lifted and placed in box

5. Conclusion

The system successfully integrates perception, motion planning, and control into a cohesive robotic manipulation pipeline. It demonstrates a complete autonomous pick-and-place task in simulation, fulfilling all key project requirements while maintaining robustness and modularity.